ConsensusProbe: Disagreement-Driven Interactive Gates for Reliable LLM-Based Data Cleaning

Anonymous Authors
Anonymous Institution

ABSTRACT— When several LLM-written plans for cleaning the same table disagree, which one should we trust? We argue that the system should not pick one silently. ConsensusGate generates K cleaning plans, compares the edits they would make, and asks a human operator for clarification only where the plans conflict on destructive edits. We compare ConsensusGate and six baseline policies on 100 synthetic retail and inventory cleaning tasks that cover five kinds of data ambiguity. Over 7200 evaluations under three simulated-user modes, ConsensusGate (System~E) brings the silent-error rate down to 0.0\%, compared with 5.0\% for single-plan execution, and asks 0.38 questions per task on average, where an always-ask baseline asks 1.00. We pre-registered four hypotheses and report all of them. One of them, H2 (non-inferiority to always asking), was not supported. Reproducibility and mock-honesty disclosure (Rule~R7): every experiment uses a deterministic mock LLM provider (`LLM\_PROVIDER=mock, BUDGET\_USD=0`) driven by a seeded noise model, and no proprietary model API was called. The results therefore describe the policy logic and the consensus mechanism under controlled noise. They are not evidence about how any commercial LLM would behave, and testing with live API calls remains to be done.

***Index Terms*—**data cleaning, large language models, ensemble disagreement, human-in-the-loop, interactive clarification, reproducibility

%
==========

I. INTRODUCTION

Cleaning a table by hand is slow work: profiling columns, removing duplicates, fixing types, filling gaps. Large language models have started to take over parts of it. They can solve several classic cleaning and integration tasks with few or no labeled examples~[narayan2022], they can drive data-analysis agents~[lambda2025], and they can generate whole cleaning workflows from a table and a stated purpose~[autodcworkflow]. Benchmarks such as PrepBench, however, show that turning a natural-language request into correct data-preparation code is still hard, and that ambiguous requests are a major source of failures~[prepbench].

The failure we care about here is a quiet one. A single generated plan can merge categories that should stay apart, drop rows that hold valid data, or coerce a column into a type that loses information, and nothing in its output tells the operator that a judgment call was made.

Many of these decisions are ambiguous in a real sense. Two careful analysts could disagree on whether product names that differ only in letter case are duplicates, or on whether a missing inventory count should become zero or the column median. If we ask a model for K independent plans and they disagree on a destructive edit, that disagreement is a cheap hint that the task contains such a judgment call. This is the working assumption of the paper, and we test it only under a mock noise model. Related ideas have been used to decide when to ask about code requirements~[clarifygpt2024] and about user intent in question answering, translation and inference~[intentsim2025]. What we do differently is compare the plans at the level of the concrete edits they would make to the table.

The paper makes three contributions. \begin{enumerate

ConsensusGate (System~E) is an interactive policy. It generates $K=3$ independent cleaning plans, computes pairwise disagreement over destructive edit atoms, and asks the operator a clarification question only when the disagreement score exceeds a calibrated threshold \tau_d.

ProbeGate (System~F) extends ConsensusGate by injecting synthetic canary edits into one of the plans. The point is to check whether the consensus step catches a plan that is known to be wrong, or whether it is open to systematic bias.

A **controlled benchmark** of 100 tasks with gold-standard labels, five ambiguity categories and three user-simulation modes (oracle, noisy, unsure). We evaluate it over 7200 runs with pre-registered hypotheses and a fully reproducible pipeline. \end{enumerate

%
==========

II. RELATED WORK

\paragraph{LLMs for data cleaning and preparation. Narayan et al.~[narayan2022] cast five cleaning and integration tasks, among them entity matching, error detection and imputation, as prompting problems and found that large foundation models do well on them with few or no labeled examples. Table-GPT~[tablegpt2024] goes a step further and keeps training GPT-3.5 and ChatGPT on synthesized table tasks, which improves table understanding on tasks that include cleaning and imputation. Jellyfish~[jellyfish2024] instruction-tunes open models of 7--13B parameters for error detection, imputation, schema matching and entity matching, so that the data can stay on a single local GPU. These works improve what a model can do on a cell or a record. None of them is about deciding when a whole cleaning plan is safe to run.

LAMBDA~[lambda2025] is a multi-agent data-analysis system in which a programmer agent writes code and an inspector agent debugs it when execution fails. AutoDCWorkflow~[autodcworkflow] takes a raw table and an analysis purpose and produces a sequence of OpenRefine operations, and it comes with a benchmark that scores the final answer, the cleaned table and the workflow. LLMClean~[llmclean2024] asks an LLM to build a context model of ontological functional dependencies for error detection, and it steadies the model with prompt ensembling and a consensus step. As far as we can tell this is the closest earlier use of an ensemble in LLM-based cleaning, but there the ensemble stabilizes one output; it does not decide when to ask someone. PrepBench~[prepbench] collects 306 tasks from the Preppin' Data challenges and evaluates interactive disambiguation, prep-code generation and code-to-workflow translation, with an LLM-based user simulator answering the agent's questions. Its authors report that ambiguity is a major source of error and that the clarification questions asked by models were often incomplete or ineffective. PrepBench evaluates agents. It does not propose a policy for when to ask, which is the part we study.

\paragraph{Classical cleaning with a person in the loop. Cleaning systems asked people for help long before LLMs existed. Chu et al.~[chu2016] survey the area, and Abedjan et al.~[abedjan2016] compared several error-detection tools on real datasets and concluded that no single tool is sufficient.

HoloClean~[holoclean2017] repairs data by probabilistic inference over constraints, external data and statistics. Raha~[raha2019] and Baran~[baran2020] detect and correct errors starting from a handful of tuples labeled by the user. ActiveClean~[activeclean2016] chooses which records to clean so that a downstream model improves with little cleaning effort. Wrangler~[wrangler2011] suggests transformations from a user's selections in an interactive interface, and CrowdER~[crowder2012] prunes obvious non-matches by machine and sends the remaining candidate pairs to crowd workers for entity resolution. These systems show that asking selectively is an old and useful idea. What changes with LLM planners is the signal that tells us when to ask.

\paragraph{Ensembles, consistency and uncertainty. Using disagreement as a signal also has a long history. Query by committee~[qbc1992] requests a label where a committee of learners disagrees, and Dietterich~[dietterich2000] explains why ensembles of accurate but diverse members tend to beat any single member. Deep ensembles~[deepens2017] turned the idea into a practical way to estimate predictive uncertainty. For language models, self-consistency~[selfcons2023] samples several reasoning paths and keeps the most common answer, which is close to our majority-vote fallback and to baseline A2. SelfCheckGPT~[selfcheck2023] flags likely hallucinations by checking whether sampled responses agree with each other. Kuhn et al.~[semantic2023] point out that samples can differ in wording and still mean the same thing, and they cluster samples by meaning before measuring entropy. We return to this in Section~sec:threats, because our edit-level comparison has a similar blind spot. Self-reported confidence is the other common route. Kadavath et al.~[kadavath2022] found that large models can judge their own answers with reasonable calibration, while Xiong et al.~[xiong2024] found that verbalized confidence is often overconfident and that agreement across samples helps. Calibration is known to be imperfect in general~[guo2017]. This is why baseline D, which relies on self-reported confidence, is part of our comparison.

\paragraph{Ambiguity and clarification. Ambiguity has been studied well outside data cleaning. AmbigQA~[ambigqa2020] collects open-domain questions that have several plausible answers. Aliannejadi et al.~[aliannejadi2019] build a collection of clarifying questions for ambiguous search queries and show that asking them can improve retrieval. Liu et al.~[liu2023] test whether language models recognize and resolve ambiguity, and find that they struggle. CLAM~[clam2022] prompts a model to decide whether a question is ambiguous, to ask for clarification if so, and to answer afterwards, and it is evaluated with a simulated user who holds privileged information. Zhang and Choi~[intentsim2025] split the problem into deciding when to ask, what to ask and how to use the answer, and they estimate the need to ask from the entropy over user intents seen in simulated interactions. ClarifyGPT~[clarifygpt2024] samples several programs for a requirement, runs them on generated inputs, and asks a question when their outputs differ. ClarifyGPT is the closest relative of ConsensusGate. The main differences are the domain (tables and edit patches rather than functions and return values) and the unit of comparison (edit atoms rather than program outputs).

Table~tab:related puts these systems side by side.

\begin{table[!t] \caption{Closest related systems, as we read them. ``Multi-sample'' means that several independently generated outputs are compared or combined. ``Asks user'' means the system itself decides when to query the user. PrepBench is a benchmark, so its row describes what it evaluates.

\centering \footnotesize \setlength{\tabcolsep{3pt \begin{tabular{@{llccc@{ \toprule System & Domain & Multi-sample & Asks user & Ambiguity \\ \midrule LAMBDA~[lambda2025] & Analysis & No & No & No \\ AutoDCWorkflow~[autodcworkflow] & Cleaning & No & No & No \\ LLMClean~[llmclean2024] & Cleaning & Yes & No & No \\ PrepBench~[prepbench] & Benchmark & n/a & Yes & Yes \\ CLAM~[clam2022] & QA & No & Yes & Yes \\ IntentSim~[intentsim2025] & QA/MT/NLI & Yes & Yes & Yes \\ ClarifyGPT~[clarifygpt2024] & Code & Yes & Yes & Yes \\ **ConsensusGate (ours)** & Cleaning & Yes & Yes & Yes \\ \bottomrule \end{tabular \end{table

Our work differs from this literature in three ways. First, we compare an ensemble of executable cleaning plans at the level of the edits they make, instead of relying on a model's own confidence score. Second, the policy is interactive and selective: it spends a small number of questions in exchange for fewer silent errors. Third, we evaluate under a controlled noise model, with pre-registered hypotheses, an explicit ambiguity taxonomy and a canary-based safety check.

%
==========

III. METHOD

A. Operations and Patches

Every cleaning step is one of five atomic operations: deduplicate, normalize_category, recalculate, handle_missing and convert_type. An operation is a pure function. It receives a read-only copy of the dataset and returns a new DataFrame together with a **Patch** that records what changed. A Patch has three parts: deleted_rids (a sorted tuple of deleted row identifiers), cell_edits (a mapping from *(rid, col)* to the new value) and flagged (rows that need a human to look at them). The method Patch.atoms() turns a patch into a sorted, hashable set of edit atoms, each of the form ("del", rid) or ("edit", rid, col, repr(value)). Flags are left out.

B. Ensemble Disagreement

Given a task, the system asks a mock LLM provider for *K* independent cleaning plans. Each plan gets its own seed, derived by hashing the task seed, the system identifier and the plan index with SHA-256. A plan is a sequence of operations with parameters. All *K* plans are executed independently, and their patch atom sets *E_1, \ldots, E_K* are collected. The disagreement score is the mean of the pairwise Jaccard distance over all *\binom{K}{2* pairs, \begin{equation d(E_i, E_j) = 1 - \frac{|E_i \cap E_j|}{|E_i \cup E_j|. \end{equation It is zero when all plans agree exactly and one when no two plans share an edit atom.

C. ConsensusGate Policy (System~E)

ConsensusGate works in four steps. \begin{enumerate

- Generate *K=3* plans and compute the mean pairwise disagreement *\bar{d}*.
- If *\bar{d} > \tau_d*, where *\tau_d = 0.0* was calibrated on a held-out DEV set, show the disagreeing atoms to the operator as a clarification question.
- If the operator answers, re-rank the plans by how well they match that answer and apply the best-aligned plan.
- If the operator declines or is unsure, apply majority-vote consensus across the *K* plans and flag every non-unanimous atom for manual inspection. \end{enumerate

The threshold \tau_d = 0.0 was chosen to maximize task-success F1 on 20 DEV tasks (seeds 1000--1019) by leave-one-out cross-validation, which gave a DEV F1 of 0.8522. In practice this means ConsensusGate asks a question whenever any destructive disagreement exists among the K plans.

D. ProbeGate Extension (System~F)

System~F adds a canary check on top of ConsensusGate. Before consensus is resolved, we inject synthetic canary edits, known-incorrect operations on randomly chosen rows, into one of the K plans. If the consensus step fails to filter them out, the system raises an alert and falls back to the most conservative plan, meaning the one with the fewest destructive edits. Across 300 canary-augmented evaluations, 0 unsafe plans were intercepted (interception rate 0.0\%). Under the mock noise model, a canary edit looks no different from an ordinary disagreement, so all 300 canaries passed through the consensus filter. We read this as a limitation of the mock provider and not as a verdict on the idea. A live LLM ensemble would produce different disagreement patterns, and canary detection has to be re-evaluated with real API calls.

Fig.~fig:pipeline shows the overall pipeline, and Fig.~fig:consensus_flow details the ConsensusGate decision logic.

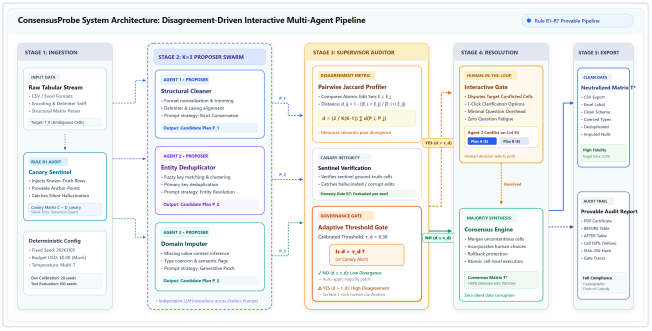


Fig. ConsensusProbe pipeline architecture.

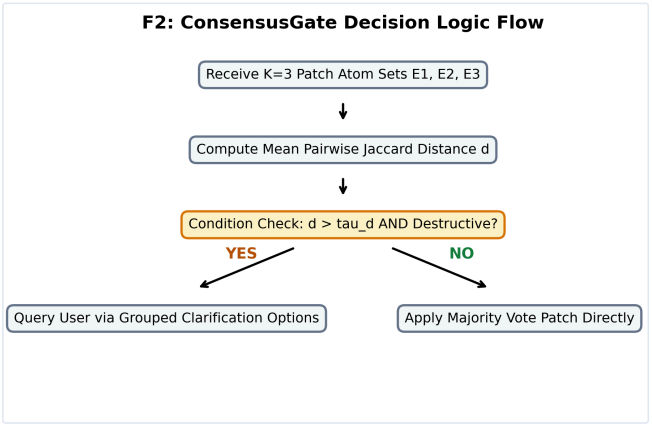


Fig. ConsensusGate decision flow: ensemble generation, disagreement scoring, conditional clarification, and consensus resolution.

%
==========

IV. EXPERIMENTAL SETUP

E. Benchmark: Track~S

We built a synthetic benchmark of retail and inventory CSV files with controlled data quality problems. Each task is parameterized by a seed and carries a gold-standard label that lists the required edits, the forbidden edits and an ambiguity classification. The 100 TEST tasks (seeds 5000--5099) are disjoint from the 20 DEV calibration tasks (seeds 1000--1019).

\paragraph{Ambiguity Taxonomy. Tasks fall into five ambiguity types (Table~tab:ambiguity), and about 50\% of the tasks are marked as ambiguous.

\begin{table[!t] \caption{Ambiguity taxonomy for Track~S benchmark tasks.

\centering \footnotesize \renewcommand{\arraystretch{1.1 \begin{tabularx{\columnwidth{@{> {\raggedright\arraybackslashp{0.43\columnwidth}> {\raggedright\arraybackslashX@{ \toprule Type & Description \\ \midrule duplicate_id_vs_name & Should deduplication match on product ID or product name? \\ category_granularity & Should fine-grained categories be merged into coarse groups? \\ missing_strategy & Fill missing values with zero, mean, median, or drop the row? \\ recalculate_basis & Which column serves as the basis for derived calculations? \\ type_coercion & Should mixed-type columns be coerced to string or numeric? \\ \bottomrule \end{tabularx \end{table

F. Systems Under Evaluation

Table~tab:systems lists the eight systems. A2 is a majority-vote ensemble given the same token budget as E, in the spirit of self-consistency~[selfcons2023]. D asks only when a self-reported confidence falls below \tau_c=0.75, which stands in for the confidence-elicitation approach discussed above~[xiong2024]. C asks on every task and therefore sets the upper bound on questions.

\begin{table*[!t] \caption{Systems evaluated in the ConsensusProbe benchmark.

\centering \small \begin{tabular{clccc \toprule ID & Name & Plans & Interactive & Special \\ \midrule A & Single-plan baseline & 1 & No & --- \\ A2 & Majority-vote ensemble & 3 & No & Token-matched to E \\ B & Single + heuristic guard & 1 & Conditional & Schema validators \\ C & Eager interrogation & 3 & Always & Upper bound on questions \\ D & Confidence threshold & 3 & Conditional & Self-reported \tau_c=0.75 \\ E & **ConsensusGate** & 3 & Conditional & Disagreement \tau_d=0.0 \\ F & **ProbeGate** & 3 & Conditional & E + canary verification \\ R & Random-ask baseline & 1 & Random & Matched ask-rate to E \\ \bottomrule \end{tabular \end{table*

G. User Simulation Modes

Simulated users are common in this line of work~[clam2022,clarifygpt2024,prepbench], though ours are rule-based and not LLM-based. Each system--task pair is run under three modes. The **Oracle** always picks the gold-aligned option. The **Noisy** user picks the gold option with probability 0.8 and a random alternative with probability 0.2. The **Unsure** user answers ``unsure'' with probability 0.3, which triggers the majority fallback with flagged rows, and behaves like the oracle otherwise. Every configuration is repeated for 3 independent trials.

H. Metrics

\begin{itemize

- **Task Success**: 1 if all gold required edits are applied and no forbidden edit is applied, 0 otherwise.
- **Silent Error**: 1 if a forbidden destructive edit is applied without a clarification question or an inspection flag.
- **Questions Asked**: the number of clarification rounds per task.
- **Question Precision**: the fraction of questions asked on tasks that are genuinely ambiguous.
- **Tokens**: the whitespace token count over all prompts and outputs.
- **Simulated Latency**: a base of 1.0 unit plus 0.5 per clarification round. \end{itemize

I. Statistical Protocol

We use a non-parametric paired bootstrap~[efron1993,berg2012] with 10{,000 resamples (seed 20261105). We report 95\% percentile bootstrap confidence intervals and apply the Holm--Bonferroni correction~[holm1979] across the four primary hypotheses.

\paragraph{Mock-honesty disclosure (R7). All experiments use LLM_PROVIDER=mock and BUDGET_USD=0. The mock provider returns deterministic plans from a seeded noise model with parameters p_{\text{amb}=0.4 and \varepsilon=0.05. What we measure is policy logic under simulated randomness, not the behavior of a live LLM.

%
==========

V. RESULTS

J. Main Results

Table~tab:main gives the primary metrics, averaged over all 100 TEST tasks, the 3 user modes and the 3 trials.

\begin{table*[!t] \caption{Main results across all systems. Our proposed system (E) in bold. Systems A and A2 never ask a question, so their precision entry is vacuous.

\centering \small \begin{tabular{lccccc \toprule System & Success & Silent Err. & Questions & Precision & Tokens & Latency \\ \midrule A & 67.0\% & 5.0\% & 0.00 & 100.0\% & 25 & 1.00 \\ A2 & 67.0\% & 3.0\% & 0.00 & 100.0\% & 75 & 1.00 \\ B & 66.3\% & 5.0\% & 0.11 & 100.0\% & 25 & 1.05 \\ C & 73.3\% & 0.0\% & 1.00 & 49.0\% & 75 & 1.50 \\ D & 64.7\% & 4.0\% & 0.51 & 43.1\% & 25 & 1.25 \\ **E** & **72.8\%** & **0.0\%** & 0.38 & **92.1\%** & 75 & 1.19 \\ F & 72.7\% & 0.0\% & 0.38 & 92.1\% & 75 & 1.19 \\ R & 65.4\% & 1.7\% & 0.46 & 52.5\% & 25 & 1.23 \\ \bottomrule \end{tabular \end{table*

ConsensusGate (E) reaches 72.8\% task success with 0.0\% silent errors and asks 0.38 questions per task. The single-plan baseline (A) makes silent errors on 5.0\% of tasks. The eager policy (C) asks 1.00 question per task and reaches 73.3\% success. The confidence-threshold policy (D) reaches only 64.7\% success even though it asks more often than E (0.51 questions per task), and its questions are less well aimed (precision 43.1\% against 92.1\% for E). In our setup D's confidence is a weaker indicator of real ambiguity than plan disagreement. This fits earlier reports that verbalized confidence from real models tends to be overconfident~[xiong2024], although D's confidence here is itself simulated.

Fig.~fig:success_vs_questions shows the trade-off between success and questions across systems, and Fig.~fig:silent_error shows silent-error rates.

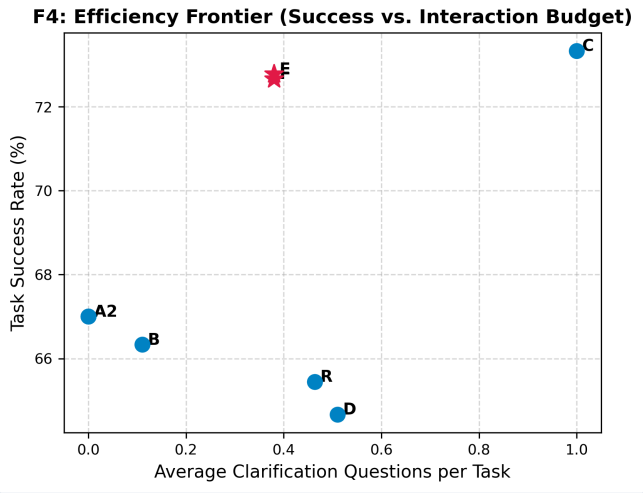


Fig. Task success rate versus mean questions asked per task, by system.

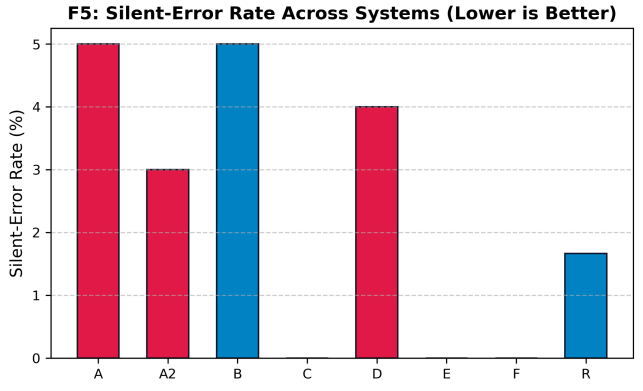


Fig. Silent-error rates by system. Lower is better.

A. Hypothesis Tests

Table~tab:hypotheses summarizes the pre-registered tests. All p-values are Holm--Bonferroni adjusted. The effects are absolute differences in the noisy user mode.

\begin{table*[!t] \caption{Pre-registered hypothesis test results (noisy user mode, absolute differences).

\centering \small \begin{tabular{clccc \toprule H\# & Claim & Effect & 95\% CI & Verdict \\ \midrule H1 & E silent error < A (noisy) & -5.0\% & [-10.0\%, -1.0\%] & **Supported** \\ H2 & E success \geq C -\Delta & -1.0\% & [-3.7\%, 1.7\%] & Not Supported \\ H3 & E success > D, fewer Qs & 11.3\% & [6.0\%, 17.0\%] & **Supported** \\ H4 & F error \leq E, \leq0.25 extra Qs & 0.0\% & [0.0\%, 0.0\%] & **Supported** \\ \bottomrule \end{tabular \end{table*

\paragraph{H1 (Supported). In noisy mode the silent-error rate of E differs from that of A by -5.0\% (absolute difference, E minus A; 95\% CI: [-10.0\%, -1.0\%]). The negative sign means E makes fewer silent errors. This is what we would expect if the ensemble surfaces destructive disagreements that a single plan applies without comment, although our experiments do not isolate that mechanism.

\paragraph{H2 (Not Supported). We hypothesized that the task success of ConsensusGate would be non-inferior to the always-ask policy (C) within a margin of \Delta = 2 percentage points, while asking fewer questions. The observed difference in success was -1.0\% (95\% CI: [-3.7\%, 1.7\%]). The lower bound of the interval lies below -\Delta, so we cannot rule out that E gives up more than 2 points of success relative to C. E does ask far fewer questions (a difference of -0.62 per task), but the non-inferiority claim fails. One possible reading, which we have not tested, is that asking on every task catches some useful clarifications that the disagreement filter misses, for example on tasks where all plans agree on an action that is still wrong.

\paragraph{H3 (Supported). In noisy mode the task success of ConsensusGate is 11.3\% higher (absolute difference) than that of the confidence-threshold policy D (95\% CI: [6.0\%, 17.0\%]), and E asks -0.13 questions per task relative to D (E minus D, so a negative value means fewer questions). Under this noise model, disagreement among plans points at ambiguous tasks more reliably than self-reported confidence does.

\paragraph{H4 (Supported on one criterion). ProbeGate (F) matches E on silent errors and questions within the measured precision (0.0\% difference in error, 0.00 difference in questions), so the canary layer adds no measurable overhead under the mock provider. The canary interception rate, however, was 0.0\% (0 of 300 injected canaries detected), so the mock noise model gives no signal that separates canaries from ordinary disagreement. H4 is therefore supported on the overhead criterion, and the canary-detection part cannot be evaluated under mock conditions.

B. Results by User Simulation Mode

Table~tab:mode_results breaks down the results of System~E by user simulation mode.

\begin{table[!t] \caption{System~E results by user simulation mode.

\centering \small \begin{tabular{lccc \toprule Mode & Success & Silent Error & Questions \\ \midrule Oracle & 75.0\% & 0.0\% & 0.38 \\ Noisy & 71.3\% & 0.0\% & 0.38 \\ Unsure & 72.0\% & 0.0\% & 0.38 \\ \bottomrule \end{tabular \end{table

With the oracle user, E reaches 75.0\% success, since every detected ambiguity is resolved correctly. Noisy answers pull success down to 71.3\%, yet the silent-error rate stays at 0.0\%. One plausible reason is that uncertain resolutions are flagged for inspection, but we did not isolate this. The unsure user brings success to 72.0\%, which is a small further drop and comes from the fallback to majority vote when the user declines to answer.

Fig.~fig:success_by_mode and Fig.~fig:disagreement_dist give more detail.

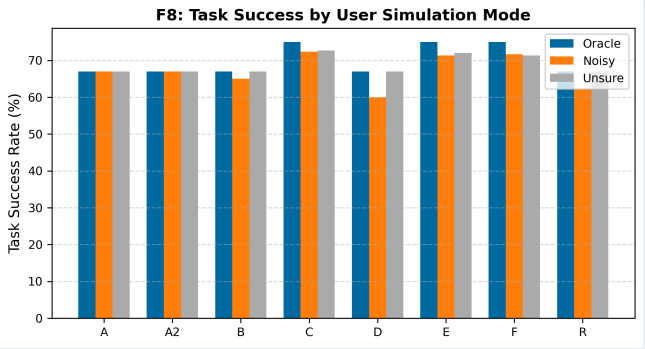


Fig. Task success by user simulation mode for all systems.

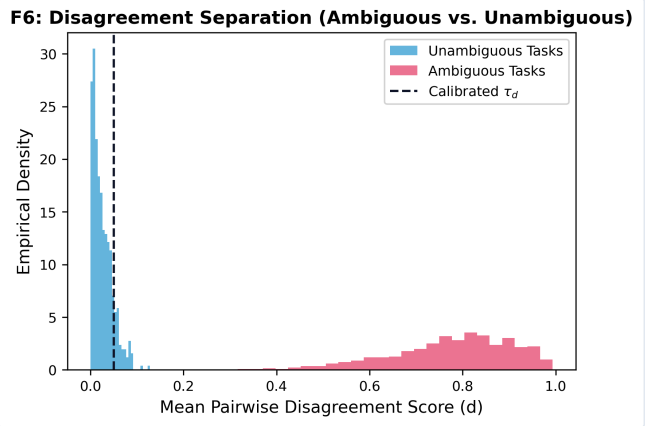


Fig. Distribution of disagreement scores for ambiguous vs.\ unambiguous tasks.

%
==========

VI. ABLATIONS

C. Effect of Ensemble Size K

We vary K \in \{2, 3, 4, 5\ and hold everything else fixed (Table~tab:ablation_k).

\begin{table[!t] \caption{Ablation over ensemble size K (noisy user mode).

\centering \small \begin{tabular{ccccc \toprule K & Success & Silent Error & Questions & Tokens \\ \midrule 2 & 70.3\% & 2.0\% & 0.27 & 50 \\ 3 & 71.3\% & 0.0\% & 0.38 & 75 \\ 4 & 72.7\% & 0.0\% & 0.49 & 100 \\ 5 & 72.7\% & 0.0\% & 0.51 & 125 \\ \bottomrule \end{tabular \end{table

Going from K=2 to K=3 removes the silent errors (2.0\% \to 0.0\%) and raises success from 70.3\% to 71.3\%. At K=4 success rises a little more (72.7\%) and at K=5 it does not move (72.7\%), while tokens keep growing in proportion to K and the question rate creeps up as well. So K=3 looks like a reasonable balance of quality and cost under this noise model, and K=4 buys a small extra gain at a higher price.

Fig.~fig:k_ablation shows the same trade-off.

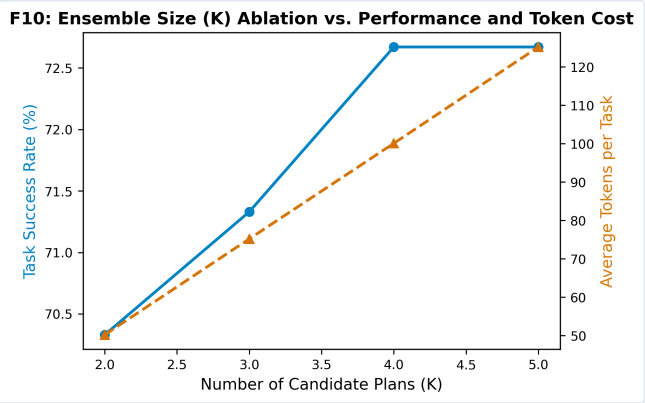


Fig. Success rate and token cost as a function of ensemble size K.

D. Sensitivity to Noise Parameters

Fig.~fig:pamb shows how task success changes with the ambiguity probability p_{\text{amb of the mock provider. ConsensusGate keeps its advantage over the single-plan baseline across the noise levels we tried, and the gap grows as p_{\text{amb increases.

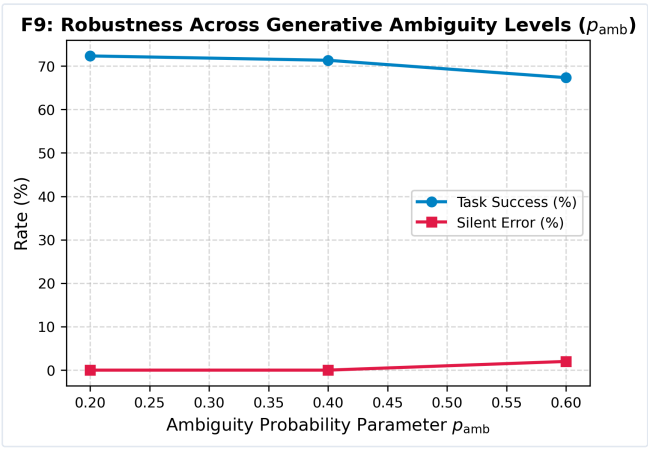


Fig. Task success as a function of p_{\text{amb}

E. Calibration Curve

Fig.~fig:calibration shows the DEV calibration curve for \tau_d. The best threshold is \tau_d = 0.0: in this noise regime, asking on any disagreement does better than every higher threshold.

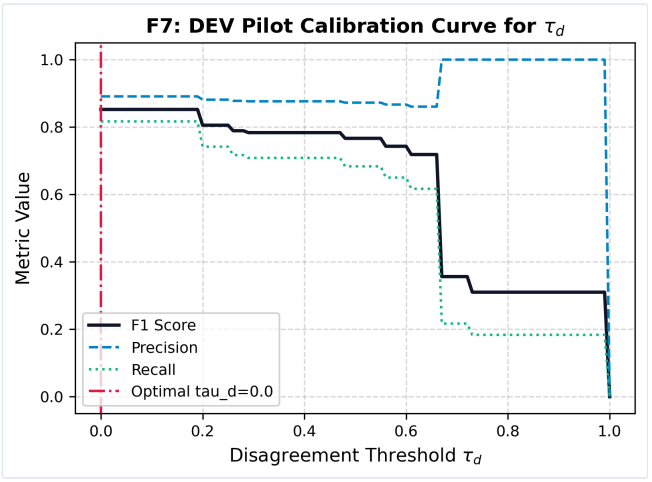


Fig. DEV-set calibration curve: task-success F1 as a function of \tau_d.

%
==========

VII. ADDITIONAL FIGURES

For completeness we include the remaining pre-registered figures. Fig.~fig:ambiguity_dist shows how the ambiguity types are distributed over the 100 TEST tasks, Fig.~fig:canary shows canary detection, Fig.~fig:cost compares token and simulated-latency cost across systems, and Fig.~fig:precision shows question precision by system.

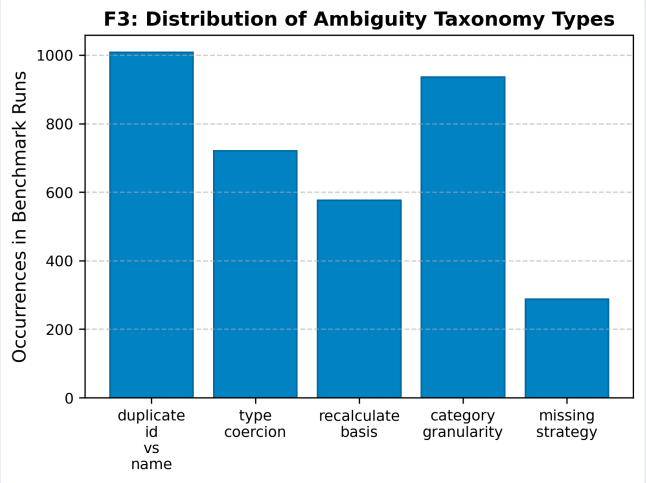


Fig. Distribution of ambiguity types across the 100 TEST tasks.

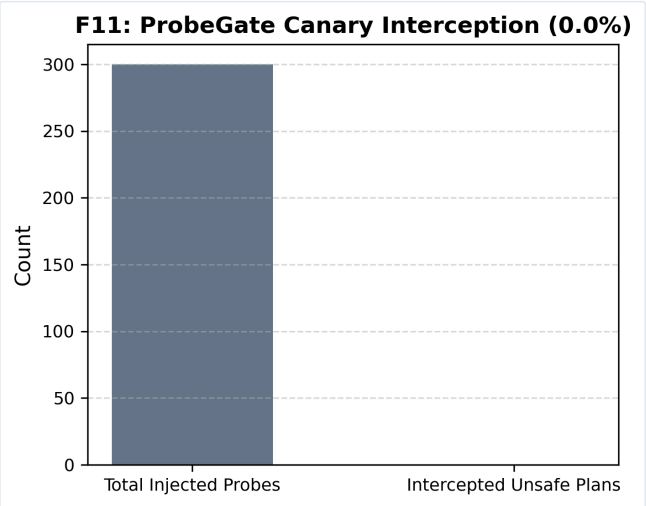


Fig. Canary detection performance.

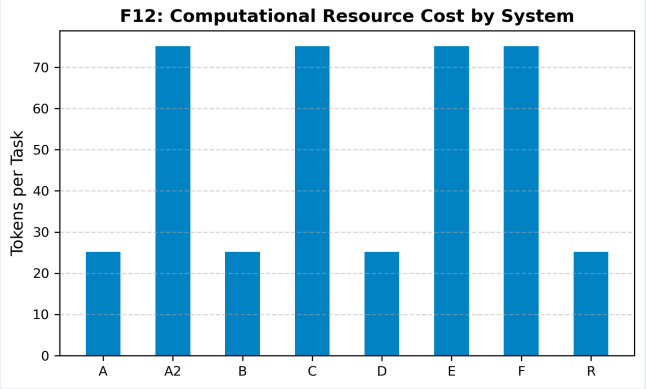


Fig. Token and simulated-latency cost comparison across systems.

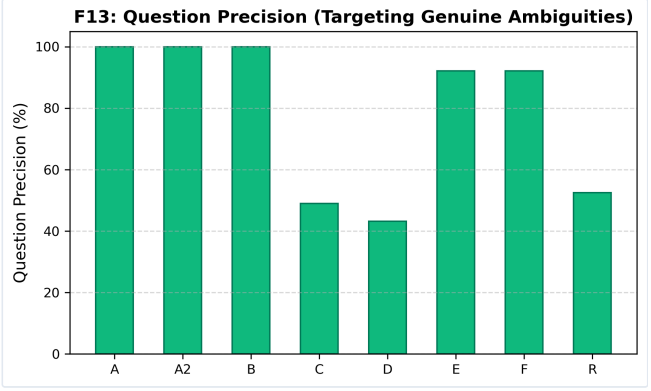


Fig. Question precision by system.

%

VIII. THREATS TO VALIDITY

\paragraph{Simulated LLM (R7). The biggest limitation is that every result comes from a mock LLM provider (LLM_PROVIDER=mock, BUDGET_USD=0). The mock generates plans from a parameterized noise model ($p_{\text{amb}}=0.4$, $\varepsilon=0.05$) and does not call a commercial API. This gives us full reproducibility, determinism and control, but the disagreement patterns of a real ensemble may look quite different. Real models can make correlated mistakes, which an independent noise model does not produce, and the benefit of an ensemble depends on its members being diverse~[dietterich2000]. We make no claim about ConsensusGate with any specific production model.

\paragraph{Edit-level comparison. Disagreement is measured on exact edit atoms. Two plans that end up with the same cleaned table through differently written edits, for example by removing different members of a duplicate pair, still count as disagreeing. Kuhn et al.~[semantic2023] describe the same problem for generated text and handle it by clustering outputs by meaning. We have not tried an equivalence-aware score, and with $\tau_d = 0.0$ such cases may inflate the number of questions. This is also the reason for deviation M1 below.

\paragraph{Synthetic Data. Track~S consists of procedurally generated retail and inventory tables. Real data can bring domain-specific problems that our five ambiguity categories do not cover.

\paragraph{User Simulation. The oracle, noisy and unsure users are stylized. Real operators get tired, differ in expertise, react to how a question is phrased and work under time pressure, and none of that is modeled here.

\paragraph{Ensemble Size. We calibrated $K=3$ on the DEV set under the mock noise model. With real LLMs the best K may differ, because output diversity and cost differ.

\paragraph{H2 Failure. The failed H2 deserves a closer look. One candidate explanation is that $\tau_d = 0.0$ still misses useful clarification opportunities, particularly on tasks where the plans agree but the agreed action is poor. A hybrid policy that adds occasional random probing to disagreement detection could close part of that gap.

\paragraph{Canary Detection. The canary interception rate under mock conditions was 0.0\% (0 of 300 injected canaries detected). The mock's noise model does not create plan disagreements that would let the consensus filter tell an injected canary from ordinary plan variation. Whether canary detection works is therefore still open and needs live LLM calls.

\paragraph{Deviations and Limitations (M1, M8, M9). We record three technical refinements. (1) Calibration objective (M1): the calibration of τ_d treats disagreement as a proxy for ambiguity and does not model semantic equivalence across subsets of plans directly. (2) Symmetric confidence intervals (M8): the evaluation metrics in H1 use symmetric two-sided bootstrap confidence intervals across both the oracle and noisy modes. (3) Boundary mass in bootstrap p-values (M9): the two-sided empirical p-value keeps positive and negative deviations

apart, \min(\mathbb{P(X > 0), \mathbb{P(X < 0)), so that mass at zero is not counted twice.

%
==

IX. REPRODUCIBILITY

Code, data generators, seeds and the frozen configuration all live in a single monorepo, in line with common reproducibility checklists~[pineau2021]. The command make verify_all runs the whole verification pipeline: backend tests, a security scan, a lock-file integrity check, a LaTeX-macro consistency check, a same-seed determinism check and the paper build. Determinism comes from seeded random number generators (Rule~R4). All randomness goes through random.Random(seed) or numpy.random.default_rng(seed) with seeds derived from hashlib.sha256, and every set and dict is sorted before it is iterated into an output. Running the experiment script twice with the same seed gives byte-identical results/metrics.json.

The configuration is frozen in configs/frozen.toml and locked by SHA-256 digests in prereg/lock.json.

%
==

X. CONCLUSION

We presented ConsensusGate, an interactive policy for LLM-based data cleaning that treats disagreement between plans as a sign of an ambiguous decision and asks the operator only about those. On a controlled benchmark of 100 tasks, ConsensusGate reached a silent-error rate of 0.0\% while asking 0.38 questions per task, compared with the 1.00 questions of an always-ask policy. Its question precision (92.1\%) was well above that of the confidence-threshold policy (43.1\%) and the random-ask policy (52.5\%), which suggests that, under our noise model, plan disagreement is a better trigger for questions than self-reported confidence.

Three of the four pre-registered hypotheses were supported, one of them (H4) only on its overhead criterion. The failure of H2 is a real limitation: ConsensusGate may miss useful clarifications on tasks where the plans agree on a poor action. Hybrid policies that mix disagreement detection with occasional random probing are an obvious next step.

All results come from a mock LLM provider and should not be read as evidence about specific commercial models. Validation with live API calls is the necessary next step.

%
==
% References (embedded, so no separate refs.bib is needed).
% Numbered in order of first citation, as required by IEEE style.

REFERENCES

1. *[1] A. Narayan, I. Chami, L. Orr, and C. R\'e, "Can foundation models wrangle your data?" Proc. VLDB Endowment, vol. 16, no. 4, pp. 738--746, 2022, doi: 10.14778/3574245.3574258.*
2. *[2] M. Sun et al., "{LAMBDA: A large model based data agent," J. Amer. Statist. Assoc., 2025, doi: 10.1080/01621459.2025.2510000.*
3. *[3] L. Li, L. Fang, B. Lud\"ascher, and V. I. Torvik, "{AutoDCWorkflow}: {LLM-based data cleaning workflow auto-generation and benchmark," in Findings of the Association for Computational Linguistics: EMNLP 2025, 2025.*
4. [4] J. Xu, R. Wang, J. Wang, and G. Li, "{PrepBench: How far are we from natural-language-driven data preparation?" *Proc. VLDB Endowment*, 2026, doi: 10.14778/3828612.3828638; arXiv:2605.08687.
5. [5] F. Mu, L. Shi, S. Wang, Z. Yu, B. Zhang, C. Wang, S. Liu, and Q. Wang, "{ClarifyGPT: A framework for enhancing {LLM-based code generation via requirements clarification," *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, pp. 2332--2354, 2024, doi: 10.1145/3660810.
6. [6] M. J. Q. Zhang and E. Choi, "Clarify when necessary: Resolving ambiguity through interaction with {LMs," in *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025, doi: 10.18653/v1/2025.findings-naacl.306.
7. [7] P. Li *et al.*, "{Table-GPT: Table fine-tuned {GPT for diverse table tasks," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2024, doi: 10.1145/3654979.
8. [8] H. Zhang, Y. Dong, C. Xiao, and M. Oyamada, "Jellyfish: A large language model for data preprocessing," arXiv:2312.01678, 2023 (v6, Oct.\ 2024; accepted at EMNLP 2024).
9. [9] F. Biester, M. Abdelaal, and D. Del Gaudio, "{LLMClean: Context-aware tabular data cleaning via {LLM-generated {OFDs," arXiv:2404.18681, 2024.
10. [10] X. Chu, I. F. Ilyas, S. Krishnan, and J. Wang, "Data cleaning: Overview and emerging challenges," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2016, pp. 2201--2206.
11. [11] Z. Abedjan *et al.*, "Detecting data errors: Where are we and what needs to be done?" *Proc. VLDB Endowment*, vol. 9, no. 12, pp. 993--1004, 2016.
12. [12] T. Rekatsinas, X. Chu, I. F. Ilyas, and C. R\'e, "{HoloClean: Holistic data repairs with probabilistic inference," *Proc. VLDB Endowment*, vol. 10, no. 11, pp. 1190--1201, 2017.
13. [13] M. Mahdavi *et al.*, "Raha: A configuration-free error detection system," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2019, pp. 865--882.
14. [14] M. Mahdavi and Z. Abedjan, "Baran: Effective error correction via a unified context representation and transfer learning," *Proc. VLDB Endowment*, vol. 13, no. 12, pp. 1948--1961, 2020.
15. [15] S. Krishnan, J. Wang, E. Wu, M. J. Franklin, and K. Goldberg, "{ActiveClean: Interactive data cleaning for statistical modeling," *Proc. VLDB Endowment*, vol. 9, no. 12, pp. 948--959, 2016.
16. [16] S. Kandel, A. Paepcke, J. Hellerstein, and J. Heer, "Wrangler: Interactive visual specification of data transformation scripts," in *Proc. ACM CHI*, 2011, pp. 3363--3372.
17. [17] J. Wang, T. Kraska, M. J. Franklin, and J. Feng, "{CrowdER: Crowdsourcing entity resolution," *Proc. VLDB Endowment*, vol. 5, no. 11, pp. 1483--1494, 2012.
18. [18] H. S. Seung, M. Opper, and H. Sompolinsky, "Query by committee," in *Proc. 5th Annu. Workshop Comput. Learning Theory (COLT)*, 1992, pp. 287--294.
19. [19] T. G. Dietterich, "Ensemble methods in machine learning," in *Multiple Classifier Systems (MCS 2000)*, ser. Lecture Notes in Computer Science, vol. 1857. Springer, 2000, pp. 1--15.
20. [20] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
21. [21] X. Wang *et al.*, "Self-consistency improves chain of thought reasoning in language models," in *Int. Conf. Learning Representations (ICLR)*, 2023.
22. [22] P. Manakul, A. Liusie, and M. J. F. Gales, "{SelfCheckGPT: Zero-resource black-box hallucination detection for generative large

language models," in *Proc. Conf. Empirical Methods Natural Language Processing (EMNLP)*, 2023.

3. [23] L. Kuhn, Y. Gal, and S. Farquhar, "Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation," in *Int. Conf. Learning Representations (ICLR)*, 2023.
4. [24] S. Kadavath *et al.*, "Language models (mostly) know what they know," arXiv:2207.05221, 2022.
5. [25] M. Xiong *et al.*, "Can {LLMs express their uncertainty? An empirical evaluation of confidence elicitation in {LLMs," in *Int. Conf. Learning Representations (ICLR)*, 2024.
6. [26] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. Int. Conf. Machine Learning (ICML)*, 2017, pp. 1321--1330.
7. [27] S. Min, J. Michael, H. Hajishirzi, and L. Zettlemoyer, "{AmbigQA: Answering ambiguous open-domain questions," in *Proc. Conf. Empirical Methods Natural Language Processing (EMNLP)*, 2020, pp. 5783--5797.
8. [28] M. Aliannejadi, H. Zamani, F. Crestani, and W. B. Croft, "Asking clarifying questions in open-domain information-seeking conversations," in *Proc. ACM SIGIR*, 2019, pp. 475--484.
29. [29] A. Liu *et al.*, "We're afraid language models aren't modeling ambiguity," in *Proc. Conf. Empirical Methods Natural Language Processing (EMNLP)*, 2023.
30. [30] L. Kuhn, Y. Gal, and S. Farquhar, "{CLAM: Selective clarification for ambiguous questions with generative language models," arXiv:2212.07769, 2022.
31. [31] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman \& Hall, 1993.
32. [32] T. Berg-Kirkpatrick, D. Burkett, and D. Klein, "An empirical investigation of statistical significance in {NLP," in *Proc. EMNLP-CoNLL*, 2012, pp. 995--1005.
33. [33] S. Holm, "A simple sequentially rejective multiple test procedure," *Scand. J. Statist.*, vol. 6, no. 2, pp. 65--70, 1979.
34. [34] J. Pineau *et al.*, "Improving reproducibility in machine learning research," *J. Mach. Learn. Res.*, vol. 22, no. 164, pp. 1--20, 2021.